

Create Database

Database Name:

bankdb

Collection Name:

Customers

Collection Name:

Transactions

Final structure:

bankdb

|

+ ----customers

|

+ ----transactions

The screenshot displays the MongoDB Data Explorer interface. On the left, a sidebar shows the cluster hierarchy: Cluster0 > bank_DB > customers. The main panel shows the 'customers' collection with 8 documents. The table view displays the following data:

	_id ObjectId	customer_id Int32	name String	city String	
1	ObjectId('6a8964e3c4c898d...')	101	"Ravi Kumar"	"Hyderabad"	
2	ObjectId('6a8965bfc4c898d...')	102	"Priya Sharma"	"Vijayawada"	
3	ObjectId('6a8965bfc4c898d...')	103	"Arjun Reddy"	"Hyderabad"	
4	ObjectId('6a8965bfc4c898d...')	104	"Sneha Rao"	"Marangal"	
5	ObjectId('6a8965bfc4c898d...')	105	"Kiran Kumar"	"Guntur"	
6	ObjectId('6a8965bfc4c898d...')	106	"Anjali Singh"	"Hyderabad"	
7	ObjectId('6a8965bfc4c898d...')	107	"Rahul Verma"	"Bengaluru"	
8	ObjectId('6a8965bfc4c898d...')	108	"Meena Devi"	"Chennai"	

PART 3: INSERT CUSTOMER DATA

Open:

```
bankdb
|
+ ----customers
|
+ ----Documents
```

Click:

```
Add Data
|
+---- Insert Document
```

Insert the following documents one by one.

Cluster0 > bank_DB > customers

Documents 8 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) Explain Reset Find Options

25 1 - 8 of 8

	_id ObjectId	customer_id Int32	name String	city String	
1	ObjectId('6a8964e3c4c890d...	101	"Ravi Kumar"	"Hyderabad"	
2	ObjectId('6a8965bfc4c890d...	102	"Priya Sharma"	"Vijayawada"	
3	ObjectId('6a8965bfc4c890d...	103	"Arjun Reddy"	"Hyderabad"	
4	ObjectId('6a8965bfc4c890d...	104	"Sneha Rao"	"Warangal"	
5	ObjectId('6a8965bfc4c890d...	105	"Kiran Kumar"	"Guntur"	
6	ObjectId('6a8965bfc4c890d...	106	"Anjali Singh"	"Hyderabad"	
7	ObjectId('6a8965bfc4c890d...	107	"Rahul Verma"	"Bengaluru"	
8	ObjectId('6a8965bfc4c890d...	108	"Meena Devi"	"Chennai"	

PART 4: INSERT TRANSACTION DATA

Open:

```
bankdb
|
+ ---- transactions
```

Click:

```
Add Data
|
+---- Insert Document
```

Cluster0 > bank_DB > transactions

Documents 15 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) [Explain](#) [Reset](#) [Find](#) [Options](#)

25 1 - 15 of 15

	_id ObjectId	transaction_id Int32	customer_id Int32	type String
1	ObjectId('6a8966e0c4c890d...	1001	101	"Deposit"
2	ObjectId('6a8966e0c4c890d...	1002	101	"Withdrawal"
3	ObjectId('6a8966e0c4c890d...	1003	102	"Deposit"
4	ObjectId('6a8966e0c4c890d...	1004	102	"Withdrawal"
5	ObjectId('6a8966e0c4c890d...	1005	103	"Deposit"
6	ObjectId('6a8966e0c4c890d...	1006	103	"Withdrawal"
7	ObjectId('6a8966e0c4c890d...	1007	104	"Deposit"
8	ObjectId('6a8966e0c4c890d...	1008	104	"Withdrawal"

QUERY 2: FIND ACTIVE CUSTOMERS

Filter:

```
{
  "status": "Active"
}
```

The screenshot shows the MongoDB Compass interface. At the top, the breadcrumb navigation is 'Cluster0 > bank_DB > customers'. Below this, the 'Documents' tab is selected, showing 8 documents. A query filter is applied: `{ city: "Hyderabad" }`. The interface includes buttons for 'View monitoring', 'Visualize Your Data', 'Generate query', 'Explain', 'Reset', 'Find', and 'Options'. Below the query bar, there are buttons for 'ADD DATA', 'BULK', 'EXPORT CODE', and a status bar showing '25' and '1 - 3 of 3'. The main area displays a table of 3 documents, all with 'city' as 'Hyderabad'.

	_id ObjectId	customer_id Int32	name String	city String	
1	ObjectId('6a8964e3c4c890d...	101	"Ravi Kumar"	"Hyderabad"	   
2	ObjectId('6a8965bfc4c890d...	103	"Arjun Reddy"	"Hyderabad"	   
3	ObjectId('6a8965bfc4c890d...	106	"Anjali Singh"	"Hyderabad"	   

QUERY 3: FIND CUSTOMERS WITH BALANCE GREATER THAN 100000

Filter:

```
{
  "balance": {
    "$gt": 100000
  }
}
```

The screenshot displays the MongoDB Atlas web interface. At the top, the breadcrumb navigation shows 'Cluster0 > bank_DB > customers'. Below this, the 'Documents' tab is active, showing a query filter: `{ "balance": { "$gt": 100000 } }`. To the right of the filter are buttons for 'Generate query', 'Explain', 'Reset', 'Find', and 'Options'. Below the query editor, there are buttons for 'ADD DATA', 'BULK', and 'EXPORT CODE'. A table of 4 documents is displayed, with columns: '_id ObjectId', 'customer_id Int32', 'name String', and 'city String'. The table contains data for four customers: Priya Sharma (Vijayawada), Sneha Rao (Warangal), Anjali Singh (Hyderabad), and Meena Devi (Chennai). The interface also includes a 'Compare tiers' button and a 'Visualize Your Data' button at the top right.

	_id ObjectId	customer_id Int32	name String	city String
1	ObjectId('6a8965bfc4c890d...	102	"Priya Sharma"	"Vijayawada"
2	ObjectId('6a8965bfc4c890d...	104	"Sneha Rao"	"Warangal"
3	ObjectId('6a8965bfc4c890d...	106	"Anjali Singh"	"Hyderabad"
4	ObjectId('6a8965bfc4c890d...	108	"Meena Devi"	"Chennai"

QUERY 4: FIND HYDERABAD CUSTOMERS

Filter:

```
{
  "city": "Hyderabad"
}
```

The screenshot shows the MongoDB Atlas web interface. At the top, there's a breadcrumb navigation: Cluster0 > bank_DB > customers. Below this, there are tabs for Documents (8), Aggregations, Schema, Indexes (1), and Validation. The Documents tab is active. In the center, there's a query editor with a filter: {"city": "Hyderabad"}. To the right of the query editor are buttons: Generate query, Explain, Reset, Find, and Options. Below the query editor, there's a toolbar with buttons: ADD DATA, BULK, EXPORT CODE, and a help icon. To the right of the toolbar are pagination controls: 25, 1 - 3 of 3, and navigation arrows. Below the toolbar, there's a table with the following columns: _id ObjectId, customer_id Int32, name String, and city String. The table contains three rows of data, all with city "Hyderabad".

	_id ObjectId	customer_id Int32	name String	city String
1	ObjectId('6a8964e3c4c890d...	101	"Ravi Kumar"	"Hyderabad"
2	ObjectId('6a8965bfc4c890d...	103	"Arjun Reddy"	"Hyderabad"
3	ObjectId('6a8965bfc4c890d...	106	"Anjali Singh"	"Hyderabad"

QUERY 5: FIND SAVINGS ACCOUNT CUSTOMERS

Filter:

```
{
  "account_type": "Savings"
}
```

Cluster0 > bank_DB > customers [View monitoring](#) [Visualize Your Data](#)

[Documents](#) 8 [Aggregations](#) [Schema](#) [Indexes](#) 1 [Validation](#)

{ "account_type": "Savings" } [Generate query](#) [Explain](#) [Reset](#) [Find](#) [Options](#)

[+ ADD DATA](#) [BULK](#) [EXPORT CODE](#) 25 1 - 5 of 5 [Refresh](#) [Previous](#) [Next](#) [Menu](#) [JSON](#) [Table](#)

customers

	_id ObjectId	customer_id Int32	name String	city String	
1	ObjectId('6a8964e3c4c890d...	101	"Ravi Kumar"	"Hyderabad"	Edit Copy Share Delete
2	ObjectId('6a8965bfc4c890d...	103	"Arjun Reddy"	"Hyderabad"	Edit Copy Share Delete
3	ObjectId('6a8965bfc4c890d...	104	"Sneha Rao"	"Warangal"	Edit Copy Share Delete
4	ObjectId('6a8965bfc4c890d...	106	"Anjali Singh"	"Hyderabad"	Edit Copy Share Delete
5	ObjectId('6a8965bfc4c890d...	108	"Meena Devi"	"Chennai"	Edit Copy Share Delete

QUERY 6: FIND TRANSACTIONS GREATER THAN 20000

Open:

transactions

|

+ ----Documents

Filter:

```
{
  "amount": {
    "$gt": 20000
  }
}
```

The screenshot shows the MongoDB Compass interface. At the top, the breadcrumb navigation is 'Cluster0 > bank_DB > transactions'. There are buttons for 'View monitoring' and 'Visualize Your Data'. Below this, the 'Documents' tab is selected, showing 15 documents. The query editor contains the following JSON query:

```
{
  "amount": {
    "$gt": 20000
  }
}
```

Buttons for 'Generate query', 'Explain', 'Reset', 'Find', and 'Options' are visible. Below the query editor, there are buttons for 'ADD DATA', 'BULK', 'EXPORT CODE', and a pagination control showing '25' and '1 - 9 of 9'. The results table is titled 'transactions' and has the following columns: '_id ObjectId', 'transaction_id Int32', 'customer_id Int32', and 'type String'. The table contains 5 rows of data:

	_id ObjectId	transaction_id Int32	customer_id Int32	type String
1	ObjectId('6a8966e0c4c890d...	1001	101	"Deposit"
2	ObjectId('6a8966e0c4c890d...	1003	102	"Deposit"
3	ObjectId('6a8966e0c4c890d...	1005	103	"Deposit"
4	ObjectId('6a8966e0c4c890d...	1007	104	"Deposit"
5	ObjectId('6a8966e0c4c890d...	1008	104	"Withdrawal"

=====

QUERY 7: FIND DEPOSITS

=====

Filter:

```
{
  "type": "Deposit"
}
```

Cluster0 > bank_DB > transactions

View monitoring

Visualize Your Data

Documents 15

Aggregations

Schema

Indexes 1

Validation

🕒

{
 "type": "Deposit"
}

Generate query ⚡

Explain

Reset

Find

Options ▶

➕ ADD DATA ▼

📄 BULK ▼

🔗 EXPORT CODE ⓘ

25 ▼

1 - 8 of 8

↺

↻

↷

☰

🔗

🏠

🏠 transactions

	_id ObjectId	transaction_id Int32	customer_id Int32	type String	
1	ObjectId('6a8966e0c4c890d...	1001	101	"Deposit"	✎ 📄 🗑
2	ObjectId('6a8966e0c4c890d...	1003	102	"Deposit"	✎ 📄 🗑
3	ObjectId('6a8966e0c4c890d...	1005	103	"Deposit"	✎ 📄 🗑
4	ObjectId('6a8966e0c4c890d...	1007	104	"Deposit"	✎ 📄 🗑
5	ObjectId('6a8966e0c4c890d...	1009	105	"Deposit"	✎ 📄 🗑
6	ObjectId('6a8966e0c4c890d...	1010	106	"Deposit"	✎ 📄 🗑

QUERY 8: FIND ATM TRANSACTIONS

Filter:

```
{
  "mode": "ATM"
}
```

Cluster0

customers

transactions

+

Cluster0 > bank_DB > transactions

[View monitoring](#) [Visualize Your Data](#)

Documents 15

Aggregations

Schema

Indexes 1

Validation

🕒

{
 "mode": "ATM"
}

[Generate query](#) [↑](#)

[Explain](#)

[Reset](#)

[Find](#)

[Options](#) ▶

[+ ADD DATA](#)

[BULK](#)

[EXPORT CODE](#)

[?](#)

25

1-7 of 7

↺

↻

↷

⋮

{ }

🏠

🏠 transactions

	_id ObjectId	transaction_id Int32	customer_id Int32	mode String	
1	ObjectId('6a8966e0c4c890d...	1002	101	"ATM"	✎ 📄 🗑️
2	ObjectId('6a8966e0c4c890d...	1004	102	"ATM"	✎ 📄 🗑️
3	ObjectId('6a8966e0c4c890d...	1006	103	"ATM"	✎ 📄 🗑️
4	ObjectId('6a8966e0c4c890d...	1008	104	"ATM"	✎ 📄 🗑️
5	ObjectId('6a8966e0c4c890d...	1011	106	"ATM"	✎ 📄 🗑️
6	ObjectId('6a8966e0c4c890d...	1013	107	"ATM"	✎ 📄 🗑️

QUERY 9: FIND DEPOSITS GREATER THAN 50000

Filter:

```
{
  "type": "Deposit",
  "amount": {
    "$gt": 50000
  }
}
```

Cluster0 > bank_DB > transactions [View monitoring](#) [Visualize Your Data](#)

[Documents](#) 15 [Aggregations](#) [Schema](#) [Indexes](#) 1 [Validation](#)

```
{
  "type": "Deposit",
  "amount": {
    "$gt": 50000
  }
}
```

[Generate query](#) [Explain](#) [Reset](#) [Find](#) [Options](#)

[ADD DATA](#) [BULK](#) [EXPORT CODE](#) [?](#)

25 1 - 3 of 3 [↺](#) [↻](#) [↷](#) [⌵](#) [⌶](#) [⌷](#)

transactions

	_id ObjectId	transaction_id Int32	customer_id Int32	type String	
1	ObjectId('6a8966e0c4c890d...	1007	104	"Deposit"	✎ 📄 📄 🗑️
2	ObjectId('6a8966e0c4c890d...	1010	106	"Deposit"	✎ 📄 📄 🗑️
3	ObjectId('6a8966e0c4c890d...	1014	108	"Deposit"	✎ 📄 📄 🗑️

◀ ▶ ⏪ ⏩

QUERY 10: FIND HYDERABAD CUSTOMERS WITH HIGH BALANCE

Filter:

```
{
  "city": "Hyderabad",
  "balance": {
    "$gt": 100000
  }
}
```

The screenshot shows the MongoDB Compass interface. At the top, there's a breadcrumb navigation: Cluster0 > bank_DB > customers. Below this, there are tabs for Documents (8), Aggregations, Schema, Indexes (1), and Validation. The Documents tab is active. In the center, a query filter is entered: `{ "city": "Hyderabad", "balance": { "$gt": 100000 } }`. To the right of the query input are buttons for [Generate query](#), [Explain](#), [Reset](#), [Find](#), and [Options](#). Below the query input, there are buttons for [ADD DATA](#), [BULK](#), and [EXPORT CODE](#). On the right side of this row, there are controls for the number of items per page (25), the total count (1-1 of 1), and navigation arrows. Below these controls, a table displays the results of the query. The table has columns: customer_id (Int32), name (String), city (String), and account_type (String). One result is shown: customer_id 106, name Anjali Singh, city Hyderabad, and account_type Savings. At the bottom, there are navigation arrows and a scrollbar.

	customer_id Int32	name String	city String	account_type String
1	106	Anjali Singh	Hyderabad	Savings

QUERY 11: FIND CUSTOMERS IN MULTIPLE CITIES

Filter:

```
{
  "city": {
    "$in": [
      "Hyderabad",
      "Vijayawada",
      "Warangal"
    ]
  }
}
```

The screenshot shows the MongoDB Compass interface. At the top, the database structure is displayed: Cluster0 > bank_DB > customers. Below this, the 'Documents' tab is active, showing 8 documents. A query filter is entered in the search bar: `{ "city": { "$in": [ "Hyderabad", "Vijayawada", "Warangal" ] } }`. The results table below shows 3 documents matching the filter. The table has columns: _id (ObjectId), customer_id (Int32), name (String), and city (String). The results are: 1. Ravi Kumar (Hyderabad), 2. Priya Sharma (Vijayawada), and 3. Arjun Reddy (Hyderabad).

	_id ObjectId	customer_id Int32	name String	city String
1	ObjectId('6a8964e3c4c890d...	101	"Ravi Kumar"	"Hyderabad"
2	ObjectId('6a8965bfc4c890d...	102	"Priya Sharma"	"Vijayawada"
3	ObjectId('6a8965bfc4c890d...	103	"Arjun Reddy"	"Hyderabad"

QUERY 12: SORT CUSTOMERS BY BALANCE

Use:

Aggregations

Add Stage:

\$sort

Enter:

```
{
  "balance": -1
}
```

The screenshot displays the MongoDB Atlas Aggregations editor for the 'customers' collection in the 'bank_DB' database. The 'Aggregations' tab is active, showing a single stage named '\$sort' with a sort key of 'balance' and a direction of '-1'. The interface includes a top navigation bar with 'Cluster0 > bank_DB > customers', 'View monitoring', and 'Visualize Your Data' buttons. Below the navigation bar, there are tabs for 'Documents' (8), 'Aggregations', 'Schema', 'Indexes' (1), and 'Validation'. The main editor area has a 'Generate aggregation' button, an 'Explain' button, a 'Run' button, and an 'Options' button. A toolbar at the bottom contains 'SAVE', 'CREATE NEW', 'EXPORT CODE', 'PREVIEW', 'STAGES', 'TEXT', 'WIZARD', and a settings icon. The 'Stage 1 \$sort' section shows the sort key configuration and an output preview of 8 documents. The preview shows a sample of the sorted data, with the first document having a balance of 320000 and the status 'Active'.

Cluster0 > bank_DB > customers

Documents 8 Aggregations Schema Indexes 1 Validation

Generate aggregation Explain Run Options

Untitled - modified SAVE CREATE NEW EXPORT CODE PREVIEW STAGES TEXT WIZARD

Stage 1 \$sort

```
1 {
2   balance: -1
3 }
```

Output preview after \$sort stage (Sample of 8 documents)

```
_id: ObjectId('6a8965bfc4c890d41f9f6bce')
customer_id: 108
name: "Meena Devi"
city: "Chennai"
account_type: "Savings"
balance: 320000
status: "Active"
```

QUERY 13: CALCULATE TOTAL TRANSACTION AMOUNT

Go to:

transactions

|

+ ---- Aggregations

Add Stage:

\$group

Enter:

```
{
  "_id": null,
  "total_amount": {
    "sum": "$balance"
  }
}
```

The screenshot shows the MongoDB Atlas Aggregations editor interface. At the top, the breadcrumb navigation indicates the path: Cluster0 > bank_DB > customers. There are buttons for 'View monitoring' and 'Visualize Your Data'. Below the breadcrumb, tabs for 'Documents' (8), 'Aggregations' (selected), 'Schema', 'Indexes' (1), and 'Validation' are visible. The main editor area has a dropdown menu showing '\$group' and a 'Generate aggregation' button. Below this, there are buttons for 'Explain', 'Run', and 'Options'. A toolbar at the bottom includes 'SAVE', 'CREATE NEW', 'EXPORT CODE', 'PREVIEW' (selected), 'STAGES', 'TEXT', 'WIZARD', and a settings icon. The main workspace is divided into two panels. The left panel shows the aggregation pipeline for 'Stage 1' with the following JSON document:

```
1 {
2   "_id": null,
3   "total_amount": {
4     "sum": "$balance"
5   }
6 }
```

The right panel, titled 'Output preview after \$group stage (Sample of 1 document)', displays the resulting JSON document:

```
_id: null
total_amount: 1120000
```

QUERY 14: CALCULATE AVERAGE TRANSACTION AMOUNT

Aggregation:

```
{
  "_id": null,
  "average_amount": {
    "avg": "balance"
  }
}
```

The screenshot shows the MongoDB Compass interface for a cluster named 'Cluster0', database 'bank_DB', and collection 'customers'. The 'Aggregations' tab is active, showing a single stage named '\$group'. The aggregation pipeline is defined as follows:

```
1 {
2   "_id": null,
3   "average_amount": {
4     "$avg": "$balance"
5   }
6 }
7
```

The interface includes buttons for 'Generate aggregation', 'Explain', 'Run', and 'Options'. Below the pipeline editor, there are options to 'SAVE', 'CREATE NEW', 'EXPORT CODE', 'PREVIEW', 'STAGES', 'TEXT', and 'WIZARD'. The 'PREVIEW' tab is selected, showing the output preview after the '\$group' stage (Sample of 1 document):

```
_id: null
average_amount : 140000
```

QUERY 15: TOTAL TRANSACTION AMOUNT BY TYPE

Aggregation:

```
[
  {
    "$group": {
      "_id": "$type",
      "total_amount": {
        "sum": "amount"
      }
    }
  }
]
```

The screenshot shows the MongoDB Compass interface for a database named 'bank_DB' and a collection named 'transactions'. The 'Aggregations' tab is selected, displaying a single aggregation pipeline stage with the following configuration:

- Group by: `$type`
- Sum of: `amount`

The results are displayed in a table with two rows:

total_amount	_id
485000	Deposit
117000	Withdrawal

The interface also includes navigation tabs for Documents (15), Aggregations, Schema, Indexes (1), and Validation. The 'Run' button is highlighted in green, indicating the aggregation has been executed successfully.

QUERY 16: NUMBER OF TRANSACTIONS PER CUSTOMER

Aggregation:

```
[
  {
    "$group": {
      "_id": "$customer_id",
      "transaction_count": {
        "$sum": 1
      }
    }
  },
]
```

Untitled - modified



+ CREATE NEW



PREVIEW



Stage 1 \$group

```
1 {
2   "_id": "$customer_id",
3   "transaction_count": {
4     "$sum": 1
5   }
6 }
```

Output preview after \$group stage (Sample of 8 documents)

```
_id: 102
transaction_count : 2
```

```
{
  "$sort": {
    "transaction_count": -1
  }
}
```

ALL RESULTS



Showing 1 - 15 count results



```
_id: ObjectId('6a8966e0c4c890d41f9f6bd5')
transaction_id: 1005
customer_id: 103
type: "Deposit"
amount: 30000
mode: "UPI"
date: "2026-08-03"
status: "Success"
```

```
_id: ObjectId('6a8966e0c4c890d41f9f6bd2')
transaction_id: 1002
customer_id: 101
type: "Withdrawal"
amount: 10000
mode: "ATM"
```

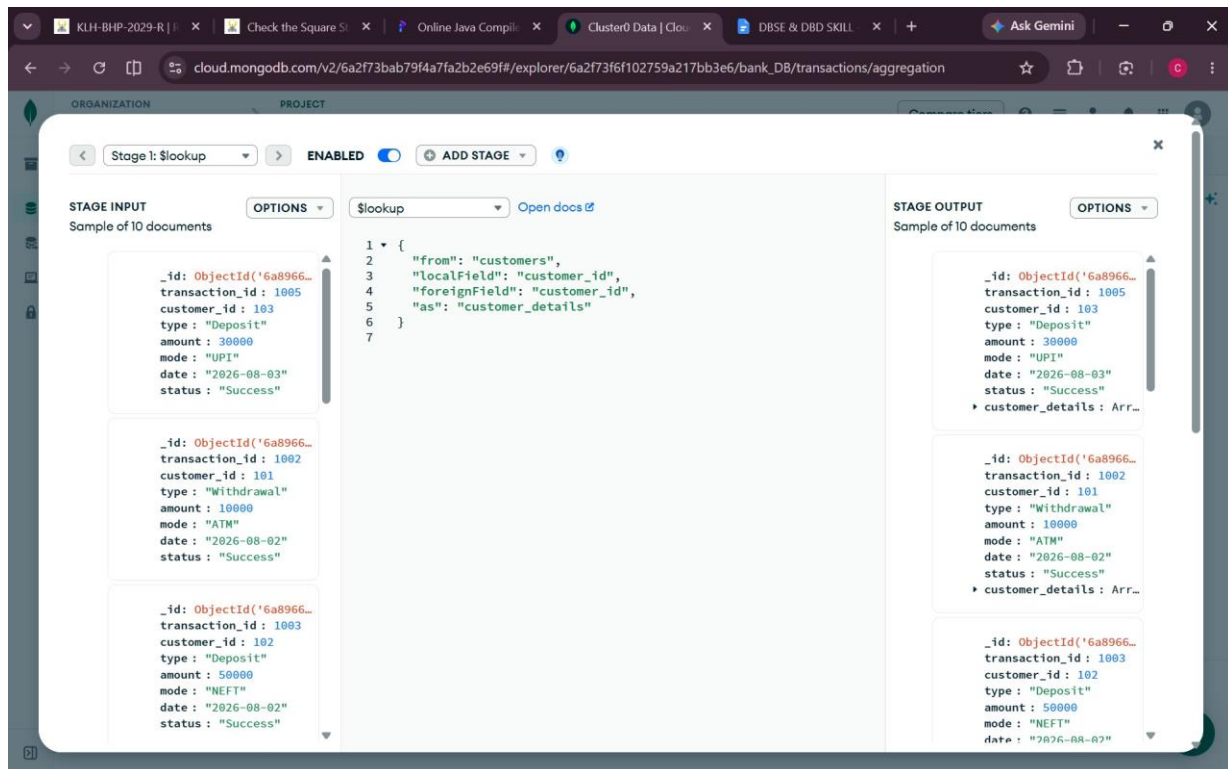
QUERY 17: CUSTOMER + TRANSACTION JOIN

Add Stage:

\$lookup

Enter:

```
{
  "from": "customers",
  "localField": "customer_id",
  "foreignField": "customer_id",
  "as": "customer_details"
}
```



QUERY 18: DISPLAY CUSTOMER NAME AND TRANSACTION AMOUNT

Use:

\$lookup

+

\$unwind

+

\$project

Pipeline:

```
[
  {
    "$lookup": {
      "from": "customers",
      "localField": "customer_id",
      "foreignField": "customer_id",
      "as": "customer"
    }
  },
  {
    "$unwind": "$customer"
  },
  {
    "$project": {
      "_id": 0,
      "transaction_id": 1,
      "customer_name": "$customer.name",
      "city": "$customer.city",
      "type": 1,
      "amount": 1,
      "mode": 1
    }
  }
]
```

Cluster0 > bank_DB > transactions

View monitoring

Visualize Your Data

Documents 15

Aggregations

Schema

Indexes 1

Validation



\$lookup

\$unwind

\$project



Explain

Run

Options

Untitled - modified



+ CREATE NEW



PREVIEW



Stage 3 \$project

```
1 {
2   "_id": 0,
3   "transaction_id": 1,
4   "customer_name": "$customer_details.name",
5   "city": "$customer_details.city",
6   "type": 1,
7   "amount": 1,
8   "mode": 1
9 }
```

Output preview after \$project stage (Sample of 10 documents)

```
transaction_id : 1005
type : "Deposit"
amount : 30000
mode : "UPI"
customer_name : "Arjun Reddy"
city : "Hyderabad"
```

QUERY 19: TOTAL TRANSACTION AMOUNT FOR EACH CUSTOMER

Pipeline:

```
[
  {
    "$group": {
      "_id": "$customer_id",
      "total_transaction": {
        "$sum": "$amount"
      }
    },
    "$lookup": {
      "from": "customers",
      "localField": "_id",
      "foreignField": "customer_id",
      "as": "customer"
    },
    "$unwind": "$customer"
  },
  {
    "$project": {
      "_id": 0,
      "customer_id": "$_id",
      "customer_name": "$customer.name",
      "total_transaction": 1
    }
  },
  {
    "$sort": {
      "total_transaction": -1
    }
  }
]
```

The screenshot displays the MongoDB Atlas pipeline editor interface. The pipeline is titled "Stage 5: \$sort" and is currently enabled. The stages listed are:

- Stage 1: \$group
- Stage 2: \$lookup
- Stage 3: \$unwind
- Stage 4: \$project
- Stage 5: \$sort (selected)

The "\$sort" stage is configured with the sort key "total_transaction" and the sort order "-1" (descending). The stage output shows a sample of 8 documents, sorted by total transaction amount in descending order:

total_transaction	customer_id	customer_name
180	108	Meena Devi
120	104	Sneha Rao
95	106	Anjali Singh
65	102	Priya Sharma
52	107	Rahul Verma

=====

QUERY 20: COMPLEX BANK TRANSACTION REPORT

=====

PIPELINE:

```
[
  {
    "$match": {
      "status": "Active",
      "balance": {
        "$gt": 80000
      }
    }
  },
  {
    "$lookup": {
      "from": "transactions",
      "localField": "customer_id",
      "foreignField": "customer_id",
      "as": "transactions"
    }
  },
  {
    "$unwind": "$transactions"
  },
  {
    "$group": {
      "_id": "$customer_id",
      "customer_name": {
        "$first": "$name"
      },
      "city": {
        "$first": "$city"
      },
      "balance": {
        "$first": "$balance"
      },
      "total_transactions": {
        "$sum": "$transactions.amount"
      },
      "transaction_count": {
        "$sum": 1
      }
    }
  },
]
```

```

{
  "$sort": {
    "total_transactions": -1
  }
}
]

```

The screenshot displays a data pipeline configuration window. At the top, a dropdown menu shows 'Stage 5: \$sort' selected. Below it, a list of stages is visible, with 'Stage 5: \$sort' checked. The main area shows the configuration for the \$sort stage, with a dropdown set to '\$sort' and a 'Open docs' link. The stage output is displayed as a sample of 6 documents, showing customer information and transaction counts.

STAGE OUTPUT
Sample of 6 documents

_id	customer_name	city	balance	total_transactions	transaction_count
102	Priya Sharma	Vijayawada	125000	6	2
108	Meena Devi	Chennai	320000	1	2
104	Sneha Rao	Warangal	210000	1	2
106	Anjali Singh	Hyderabad	175000	9	2